CMP project 4

Karmukilan Somasundaram

March 2026

1 Introduction

Our goal for this project is to use the nearly free electron model to plot the electron band structure for the following weak potential

$$V(x) = A\cos\frac{2\pi x}{a'} + \frac{A}{2}\cos\frac{4\pi x}{a'} + \frac{A}{3}\cos\frac{6\pi x}{a'} \tag{1}$$

where A=3eV and a'=a/2.

2 Band structure and Density of states,convergence and Fermi Energy

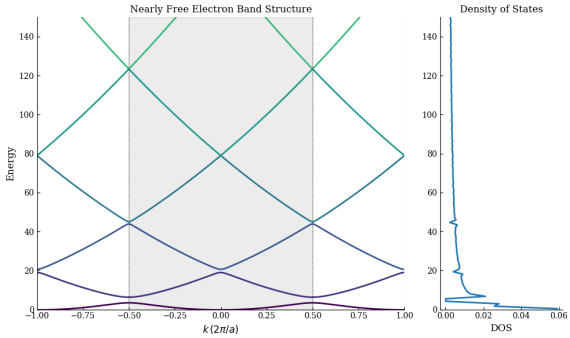

Figure 1: Band structure and density of states for V(x)

We can observe that the density of states distributes itself by moving away from the ground state as we increase the truncation parameter

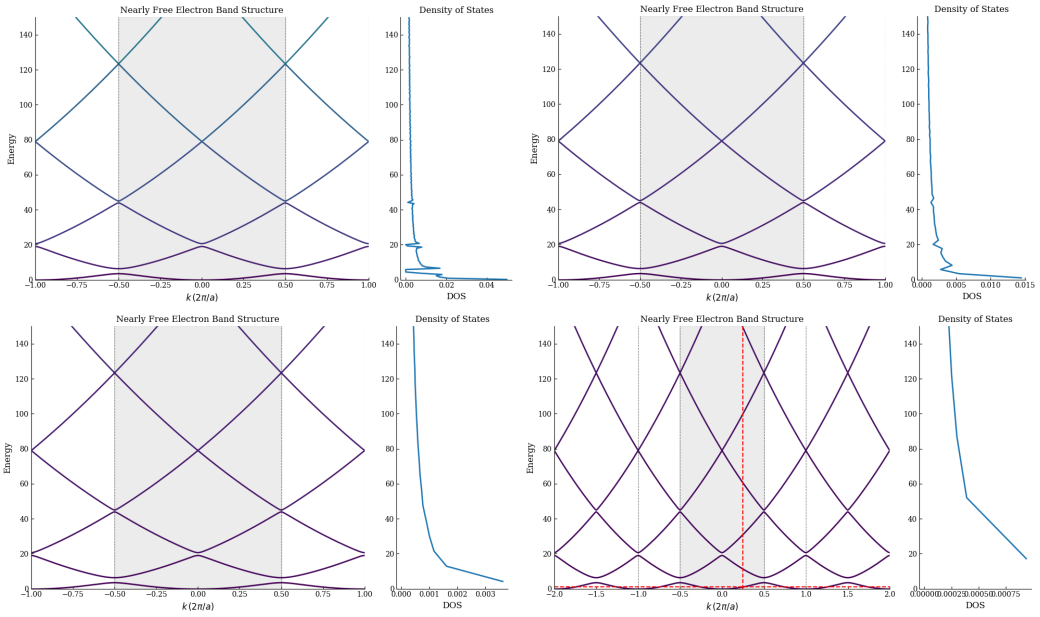

Figure 2: a.10, b.20, c.40, d.80 where the number denotes the number of oscillatory modes

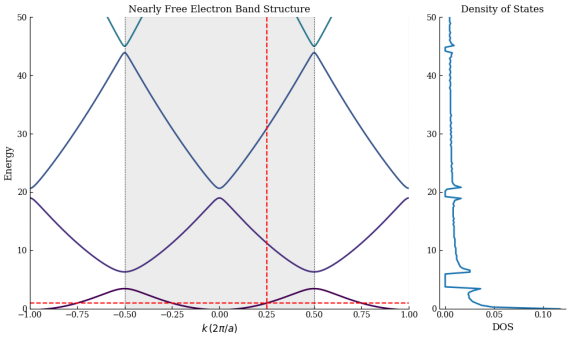

Figure 3: Fermi energy as N approaches infinity

3 2-Folding

The new weak potential is

$$V(x) = 2A\cos\frac{\pi x}{a'} + A\cos\frac{2\pi x}{a'} + \frac{A}{2}\cos\frac{4\pi x}{a'} + \frac{A}{3}\cos\frac{6\pi x}{a'} \tag{2}$$

We could now change $a'' = a'/2$ and proceed further which yields

$$V(x) = 2A\cos\frac{2\pi x}{a''} + A\cos\frac{4\pi x}{a''} + \frac{A}{2}\cos\frac{8\pi x}{a''} + \frac{A}{3}\cos\frac{12\pi x}{a''} \tag{3}$$

From this we can carry our simulation

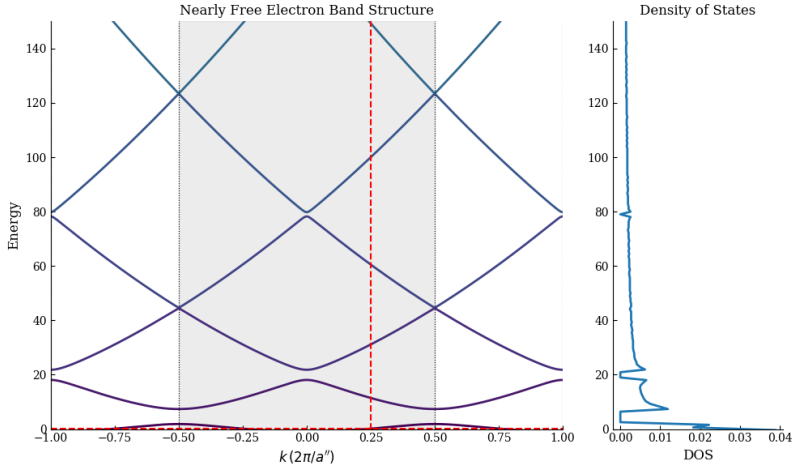

Figure 4: Folded Band structure

4 Total Energy per unit cell

This 5 plot agrees with the fact that the asymmetry induced by the potential reduces the total energy of the system (Spontaneous Symmetry breaking)

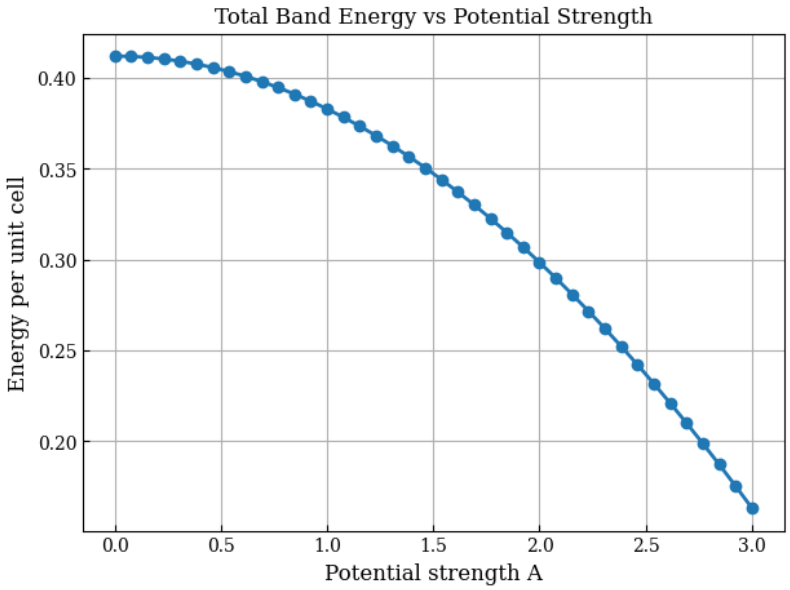

Figure 5: E_{tot} vs A

5 Python Codes

Listing 1: Band structure,Density Of States

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams.update({
    "font.family": "serif",
    "axes.labelsize": 12,
    "xtick.direction": "in",
    "ytick.direction": "in",
})

def nfe_band_and_dos():

    a_ = 1.0
    G0 = 2*np.pi/a_

    # First two Brillouin zones
    k_min, k_max = -2, 2
    num_k_points = 1000

    n = int(input("Truncation parameter n: "))
    v_input = input(f"Enter {n} values for V_G (comma separated): ")

```

```
    v_list = [float(x.strip()) for x in v_input.split(',')]

    while len(v_list) < 2*n:
        v_list.append(0.0)

    g_indices = np.arange(-n, n+1)
    num_basis = len(g_indices)

    k_vals = np.linspace(k_min*G0, k_max*G0, num_k_points)

    energies_grid = []

    # Construct Hamiltonian and diagonalize
    for k in k_vals:

        H = np.zeros((num_basis, num_basis))

        for i in range(num_basis):
            for j in range(num_basis):

                if i == j:
                    H[i,j] = 0.5*(k - g_indices[i]*G0)**2

                else:
                    m = abs(g_indices[i]-g_indices[j])
                    if m <= len(v_list):
                        H[i,j] = v_list[m-1]

        energies_grid.append(np.linalg.eigvalsh(H))

    energies_grid = np.array(energies_grid)

    # =========================
    # Density of States
    # =========================

    energies_flat = energies_grid.flatten()

    bins = 1000
    dos, energy_bins = np.histogram(energies_flat, bins=bins, density=True)

    energy_centers = 0.5*(energy_bins[1:] + energy_bins[:-1])

    # =========================
    # Plotting
    # =========================

    fig = plt.figure(figsize=(10,6))

    gs = fig.add_gridspec(1,2,width_ratios=[3,1])
```

```
73
74      ax_band = fig.add_subplot(gs[0])
75      ax_dos = fig.add_subplot(gs[1], sharey=ax_band)
76
77      colors = plt.cm.viridis(np.linspace(0,0.8,num_basis))
78
79      for i in range(num_basis):
80          ax_band.plot(k_vals/G0, energies_grid[:,i], color=colors[i], lw=2)
81
82      ax_band.axvspan(-0.5,0.5,color='gray',alpha=0.15)
83
84      for boundary in [-1,-0.5,0.5,1]:
85          ax_band.axvline(boundary,color='black',linestyle=':',lw=0.8)
86
87      kF = np.pi/(2*a_)
88
89  # find closest k-point
90      k_index = np.argmin(np.abs(k_vals - kF))
91
92  # first band energy
93      E_F = energies_grid[k_index,0]
94
95
96      ax_band.set_xlim(k_min,k_max)
97      ax_band.set_ylim(0,150)
98
99      ax_band.set_xlabel(r"$k\,(2\pi/a)$")
100     ax_band.set_ylabel("Energy")
101     ax_band.axhline(E_F, color='red', linestyle='--', label="Fermi Energy")
102     ax_band.axvline(kF/G0, color='red', linestyle='--')
103
104     ax_band.set_title("Nearly Free Electron Band Structure")
105
106     # DOS plot
107     ax_dos.plot(dos, energy_centers, lw=2)
108
109     ax_dos.set_xlabel("DOS")
110     ax_dos.set_title("Density of States")
111
112     ax_dos.spines['top'].set_visible(False)
113     ax_dos.spines['right'].set_visible(False)
114
115     ax_band.spines['top'].set_visible(False)
116     ax_band.spines['right'].set_visible(False)
117
118     plt.tight_layout()
119     plt.show()
120
121 nfe_band_and_dos()
122
```

```
[style=python,caption=
{Total Energy}]
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams.update({
    "font.family": "serif",
    "axes.labelsize": 12,
    "xtick.direction": "in",
    "ytick.direction": "in",
})

def nfe_band_and_dos():

    a_ = 1.0
    G0 = 2*np.pi/a_

    # First two Brillouin zones
    k_min, k_max = -2, 2
    num_k_points = 1000

    n = int(input("Truncation parameter n: "))
    v_input = input(f"Enter {n} values for V_G (comma separated): ")

    v_list = [float(x.strip()) for x in v_input.split(',')]

    while len(v_list) < 2*n:
        v_list.append(0.0)

    g_indices = np.arange(-n, n+1)
    num_basis = len(g_indices)

    k_vals = np.linspace(k_min*G0, k_max*G0, num_k_points)

    energies_grid = []

    # Construct Hamiltonian and diagonalize
    for k in k_vals:

        H = np.zeros((num_basis, num_basis))

        for i in range(num_basis):
            for j in range(num_basis):

                if i == j:
                    H[i,j] = 0.5*(k - g_indices[i]*G0)**2

                else:
                    m = abs(g_indices[i]-g_indices[j])
                    if m <= len(v_list):
```

```
                        H[i,j] = v_list[m-1]

        energies_grid.append(np.linalg.eigvalsh(H))

    energies_grid = np.array(energies_grid)

    # =========================
    # Density of States
    # =========================

    energies_flat = energies_grid.flatten()

    bins = 1000
    dos, energy_bins = np.histogram(energies_flat, bins=bins, density=True)

    energy_centers = 0.5*(energy_bins[1:] + energy_bins[:-1])

    # =========================
    # Plotting
    # =========================

    fig = plt.figure(figsize=(10,6))

    gs = fig.add_gridspec(1,2,width_ratios=[3,1])

    ax_band = fig.add_subplot(gs[0])
    ax_dos = fig.add_subplot(gs[1], sharey=ax_band)

    colors = plt.cm.viridis(np.linspace(0,0.8,num_basis))

    for i in range(num_basis):
        ax_band.plot(k_vals/G0, energies_grid[:,i], color=colors[i], lw=2)

    ax_band.axvspan(-0.5,0.5,color='gray',alpha=0.15)

    for boundary in [-1,-0.5,0.5,1]:
        ax_band.axvline(boundary,color='black',linestyle=':',lw=0.8)

    kF = np.pi/(2*a_)

# find closest k-point
    k_index = np.argmin(np.abs(k_vals - kF))

# first band energy
    E_F = energies_grid[k_index,0]


    ax_band.set_xlim(k_min,k_max)
    ax_band.set_ylim(0,150)

```

```
    ax_band.set_xlabel(r"$k\,(2\pi/a)$")
    ax_band.set_ylabel("Energy")
    ax_band.axhline(E_F, color='red', linestyle='--', label="Fermi Energy")
    ax_band.axvline(kF/G0, color='red', linestyle='--')

    ax_band.set_title("Nearly Free Electron Band Structure")

    # DOS plot
    ax_dos.plot(dos, energy_centers, lw=2)

    ax_dos.set_xlabel("DOS")
    ax_dos.set_title("Density of States")

    ax_dos.spines['top'].set_visible(False)
    ax_dos.spines['right'].set_visible(False)

    ax_band.spines['top'].set_visible(False)
    ax_band.spines['right'].set_visible(False)

    plt.tight_layout()
    plt.show()

nfe_band_and_dos()
```